Advanced Day 6 — Session 6 (Hours 21–24)

Python Programming (Advanced) • GUI CRUD Wiring, Architecture, and Milestone Readiness

Session 6 Overview

Topics Covered Today

- Hour 21 — CRUD wiring: update and delete workflows in the GUI
- Hour 22 — Controller separation and callback cleanup
- Hour 23 — JSON persistence and GUI polish sprint
- Hour 24 — Checkpoint 3: GUI milestone demo

Session Goal

Turn a promising interface into a **trustworthy mini-application**.
Keep stable IDs as the bridge between UI state and domain operations.

Source Alignment

Scope Guardrails for Today

In Scope

- Update and delete wired to stable record IDs
- Tkinter GUI architecture with `App` / controller class
- JSON load on startup and save after every CRUD change
- Graceful error handling with `messagebox`
- Checkpoint 3 demo: add → list → update → delete → restart → load

Not Yet

- SQLite or any database backend (Session 7)
- Async or multi-threaded GUI patterns
- Authentication or production security

Hour 21 — Update and Delete Workflows

Learning Outcomes

- Implement **select** → **load** → **edit** → **save update** workflow
- Delete with confirmation via `messagebox.askyesno`
- Refresh the display after every mutation
- Handle `NotFoundError` and `ValidationError` gracefully
- Target records by **stable ID**, never by list index

Source Alignment

- `Advanced/lessons/lecture/Day6_Hour1_Advanced.md` → `## Learning objectives`

Why Update/Delete Are Harder Than Add

Create	Update / Delete
Read form	Identify the correct record
Validate	Keep selection state accurate
Save new record	Avoid stale widgets after refresh
Refresh list	Recover if the record is already gone

> Display order is not identity. Stable IDs are.

- **Fragile pattern** — `records[selected_index] = ...`
- **Durable pattern** — `service.update_item(selected_id, ...)`

Source Alignment

- `Advanced/lessons/lecture/Day6_Hour1_Advanced.md` → `### 2) Why add is easy`

Stable IDs: The Core Design Choice

A stable ID is created **once**, when the record is created.

- Does **not** change when the record is edited
- Does **not** depend on screen or sort order
- Does **not** depend on list filtering
- **Survives** save/load cycles

Recommended form — a string UUID:

```
from uuid import uuid4

item_id = str(uuid4())  # e.g. "3f2b1c4e-..."
```

Keep `self.selected_item_id: str | None = None` in the `App` class.
Capture it on selection. Pass it to every service call.

Domain Model: TrackerItem + TrackerService

```python
from dataclasses import dataclass
from uuid import uuid4


@dataclass(slots=True)
class TrackerItem:
    item_id: str
    name: str
    category: str
    status: str
    notes: str = ""

    @classmethod
    def create(cls, name, category, status, notes=""):
        return cls(item_id=str(uuid4()), name=name,
                   category=category, status=status, notes=notes)
```

Service enforces validation and owns mutation:

Selection Handler: Capturing the Stable ID

```python
def on_item_selected(self, event=None):
    selection = self.tree.selection()
    if not selection:
        self.selected_item_id = None
        return

    values = self.tree.item(selection[0], "values")
    item_id = values[0]          # first column stores the stable ID
    self.selected_item_id = item_id

    try:
        item = self.service.get_item(item_id)
    except NotFoundError as error:
        messagebox.showerror("Selection Error", str(error))
        self.refresh_items()
        self.clear_form()
        return

    self.name_var.set(item.name)
    self.category_var.set(item.category)
    self.status_var.set(item.status)
    self.notes_text.delete("1.0", tk.END)
    self.notes_text.insert("1.0", item.notes)
```

Update Handler

```
def on_update(self):
    if self.selected_item_id is None:
        messagebox.showwarning("No Selection",
                               "Please select a record to update.")
        return

    try:
        updated = self.service.update_item(
            item_id=self.selected_item_id,
            name=self.name_var.get(),
            category=self.category_var.get(),
            status=self.status_var.get(),
            notes=self.notes_text.get("1.0", tk.END).strip(),
        )
    except ValidationError as error:
        messagebox.showerror("Validation Error", str(error))
        return
    except NotFoundError as error:
        messagebox.showerror("Update Failed", str(error))
        self.refresh_items()
        self.clear_form()
        self.selected_item_id = None
        return

    self.refresh_items(selected_item_id=updated.item_id)
    self.set_status(f"Updated: {updated.name}")
```

Delete Flow and Confirmation

The delete checklist — always follow this order:

1. Guard: ensure a record is actually selected
2. Fetch item name to show in the confirmation prompt
3. Ask with `messagebox.askyesno` — destructive actions need explicit consent
4. Call `service.delete_item(selected_id)`
5. Refresh the list from the service layer
6. Clear stale form values and stale `selected_item_id`
7. Show status feedback

```
confirmed = messagebox.askyesno(
    title="Confirm Delete",
    message=f"Delete '{item.name}'? This action cannot be undone right now.",
)
if not confirmed:
```

Delete Handler

```python
def on_delete(self):
    if self.selected_item_id is None:
        messagebox.showwarning("No Selection", "Select a record to delete.")
        return

    try:
        item = self.service.get_item(self.selected_item_id)
    except NotFoundError as error:
        messagebox.showerror("Delete Failed", str(error))
        self.refresh_items()
        self.clear_form()
        self.selected_item_id = None
        return

    if not messagebox.askyesno("Confirm Delete",
                               f"Delete '{item.name}'?"):
        return

    try:
        self.service.delete_item(item.item_id)
    except NotFoundError as error:
        messagebox.showerror("Delete Failed", str(error))
        return

    self.refresh_items()
    self.clear_form()
    self.selected_item_id = None
    self.set_status(f"Deleted: {item.name}")
```

Refresh Pattern: One Source of Truth

Rebuild the list from the service layer every time data changes:

```
def refresh_items(self, selected_item_id=None):
    for row_id in self.tree.get_children():
        self.tree.delete(row_id)

    for item in self.service.list_items():
        row_id = self.tree.insert(
            "", tk.END,
            values=(item.item_id, item.name, item.category, item.status),
        )
        if selected_item_id == item.item_id:
            self.tree.selection_set(row_id)
            self.tree.focus(row_id)
```

Any code path that changes data calls **one** refresh method.

Do not patch visual details by hand — rebuild from the service.

Lab: GUI Update/Delete Round Trip

Time — 25–35 minutes

Tasks

1. Wire row selection so the form loads the selected record
2. Track the selected record's stable ID
3. Implement **Update** — send changes to the service layer
4. Implement **Delete** with confirmation dialog
5. Refresh the list automatically after each change
6. Handle `NotFoundError` and `ValidationError` with user-facing messages

Completion Criteria

✓ Update works reliably and targets the correct record
✓ Delete works reliably with confirmation

Common Pitfalls — Hour 21

⚠️ **Using list index as the record ID** — breaks after insert, delete, or sort
⚠️ **Mutating stale widget state** — always reload from the service after any change
⚠️ **Deleting while pointing at a stale selection** — guard with an early nil-check
⚠️ **Forgetting to refresh after update/delete** — UI falls out of sync silently
⚠️ **Suppressing** `NotFoundError` — stale IDs are a design signal, not noise

Quick Check

> Why is a stable ID better than relying on a visual row index?

Expected answer: row positions change after insertions, deletes, and re-sorts.
An ID created once at record creation survives all of those events.

Hour 22 — GUI Architecture and Controller Separation

Learning Outcomes

- Recognize the symptoms of callback spaghetti
- Refactor toward a `TrackerApp` controller class
- Keep callbacks thin — collect input, call service, update screen
- Separate widget construction from event-handling logic
- Manage `selected_item_id` and status state in one clear place
- Avoid circular imports by enforcing a one-way dependency direction

Source Alignment

- `Advanced/lessons/lecture/Day6_Hour2_Advanced.md` → `## Learning objectives`

What Callback Spaghetti Looks Like

Common symptoms in a growing GUI codebase:

- Many top-level functions with **unclear ownership**
- Globals for widgets: `name_entry`, `tree`, `status_label`
- Duplicate form-reading logic in every handler
- Duplicate refresh logic after every action
- **Business rules embedded directly in callbacks**
- Mysterious bugs from stale selection state
- Hard-to-test code because logic is fused to UI objects

The smell test

> If adding **one button** means editing six scattered unrelated functions, the problem is structure, not Tkinter.

Before: Global-State Mess

```
service = TrackerService()
selected_item_id = None        # global — shared, unmanaged

def update_item():
    global selected_item_id
    name = name_entry.get()              # repeated form reads
    category = category_entry.get()
    status = status_combo.get()
    notes = notes_text.get("1.0", "end").strip()
    service.update_item(selected_item_id, name, category, status, notes)
    refresh_tree()              # duplicated in add_item and delete_item
    status_label.config(text="Updated item")

def delete_item():
    global selected_item_id
    service.delete_item(selected_item_id)   # no confirmation, no error guard
    refresh_tree()
    clear_form()
```

Target Architecture

```
main.py
  └── assembles TrackerApp(root, service)

ui/app.py   ← TrackerApp
  ├── build_ui()          widget creation
  ├── get_form_data()     read form once, reuse everywhere
  ├── clear_form()
  ├── refresh_items()
  ├── on_add()            thin callbacks → service calls
  ├── on_update()
  ├── on_delete()
  ├── on_item_selected()
  └── set_status()

core/service.py   ← validation, mutation, lookup
core/models.py    ← TrackerItem
core/exceptions.py
```

> The `App` class is a **traffic director**, not the whole city.

TrackerApp: Class Shell

```python
import tkinter as tk
from tkinter import ttk, messagebox
from core.exceptions import NotFoundError, ValidationError
from core.service import TrackerService


class TrackerApp:
    def __init__(self, root: tk.Tk, service: TrackerService) -> None:
        self.root = root
        self.service = service
        self.selected_item_id: str | None = None
        self.status_text = tk.StringVar(value="Ready")
        self.name_var = tk.StringVar()
        self.category_var = tk.StringVar()
        self.status_var = tk.StringVar()

        self.root.title("Full-Stack Tracker")
        self.build_ui()
        self.refresh_items()
```

build_ui(): Widget Construction in One Place

```python
def build_ui(self) -> None:
    f = ttk.Frame(self.root, padding=12)
    f.grid(row=0, column=0, sticky="nsew")
    self.root.columnconfigure(0, weight=1)
    self.root.rowconfigure(0, weight=1)

    ttk.Label(f, text="Name").grid(row=0, column=0, sticky="w")
    ttk.Entry(f, textvariable=self.name_var, width=30).grid(row=0, column=1, sticky="ew")

    ttk.Label(f, text="Status").grid(row=1, column=0, sticky="w")
    self.status_combo = ttk.Combobox(
        f, textvariable=self.status_var,
        values=["Open", "In Progress", "Done"], state="readonly",
    )
    self.status_combo.grid(row=1, column=1, sticky="ew")

    bf = ttk.Frame(f)
    bf.grid(row=3, column=0, columnspan=2, sticky="ew", pady=8)
    ttk.Button(bf, text="Add",    command=self.on_add).grid(row=0, column=0, padx=4)
    ttk.Button(bf, text="Update", command=self.on_update).grid(row=0, column=1, padx=4)
    ttk.Button(bf, text="Delete", command=self.on_delete).grid(row=0, column=2, padx=4)

    self.tree = ttk.Treeview(f, columns=("id","name","category","status"),
                             show="headings", height=10)
    self.tree.grid(row=4, column=0, columnspan=2, sticky="nsew")
    self.tree.bind("<<TreeviewSelect>>", self.on_item_selected)
```

Thin Callbacks: get_form_data + on_add

Extract repeated form-reading into **one** helper:

```
def get_form_data(self) -> dict[str, str]:
    return {
        "name":     self.name_var.get(),
        "category": self.category_var.get(),
        "status":   self.status_var.get(),
        "notes":    self.notes_text.get("1.0", tk.END).strip(),
    }
```

Callback now reads like a policy statement:

```
def on_add(self) -> None:
    try:
        item = self.service.add_item(**self.get_form_data())
    except ValidationError as error:
        messagebox.showerror("Validation Error", str(error))
        return
```

State Management: Selection and Status

Keep cleanly named app state on `self`:

- `self.selected_item_id` — the one piece of identity state
- `self.status_text` — a `StringVar` the status label binds to
- `self.name_var`, `self.category_var`, `self.status_var` — form mirrors

Avoid hidden state:

- globals changed from multiple functions
- relying on whatever tree row still exists after refresh
- leaving form fields populated with unknown record content

`set_status` stays trivial:

```
def set_status(self, message: str) -> None:
    self.status_text.set(message)
```

Module Boundaries and Import Direction

```
main.py           assembles dependencies
  ↓ imports
ui/app.py         imports service + exceptions
  ↓ imports
core/service.py   imports models + exceptions
  ↓ imports
core/models.py    no imports from ui or service
```

Rule — `core/service.py` must **never** import `ui/app.py`.

If that boundary breaks:

1. Step back before debugging behavior
2. Re-establish one-way dependency
3. Use dependency injection to pass objects rather than importing

Source Alignment

Refactor Strategy: Move One Piece at a Time

Safe sequence — never rewrite the whole GUI from scratch:

1. **Freeze behavior** — run the app once; record what works; commit or backup
2. **Create the class shell** — open a window from `TrackerApp.__init__`
3. **Move widget creation** into `build_ui()`; verify window still opens
4. **Move one callback** (Add first); test immediately
5. **Extract helpers** — `get_form_data()`, `clear_form()`, `refresh_items()`
6. **Move remaining callbacks** — Update, Delete, selection handler
7. **Verify** behavior is unchanged end-to-end
8. **Split modules** only after the class is stable

> Behavior must be preserved at every step.
> A refactor that breaks working features is a regression, not progress.

Lab: Refactor Without Breaking Behavior

Time — 25–35 minutes

Required Tasks

- Create a `TrackerApp` class that builds and owns the interface
- Move widget references into `build_ui()` on `self`
- Move event handlers into methods on the class
- Extract `get_form_data()` and `refresh_items()` helpers
- Keep add/update/delete/select behavior working correctly
- Confirm business rules remain in the service layer

Completion Criteria

✓ New teammate can locate widget creation and callback logic quickly
✓ App launches and performs CRUD correctly after refactor

Common Pitfalls — Hour 22

⚠️ **Over-refactoring midstream** — move one thing, test, then move the next
⚠️ **Callback spaghetti inside a class** — a class wrapper does not fix scattered logic
⚠️ **Circular imports** — if `service.py` imports `app.py`, re-establish one-way deps
⚠️ **Controller as the business brain** — keep validation and domain logic in the service
⚠️ **Giant `build_ui()` method** — extract `build_form()`, `build_buttons()`, `build_list()`

Quick Check

> What is one sign that your GUI code needs refactoring?

Strong answer: "*If adding one button requires editing five scattered globals or functions, the structure is the problem.*"

Hour 23 — Persistence and Polish Sprint

Learning Outcomes

- Load tracker data from JSON when the app starts
- Save after add, update, and delete operations
- Handle missing files and malformed JSON without crashing
- Add at least one small usability improvement
- Explain why JSON is the right persistence step before SQLite
- Prepare a GUI milestone demonstrable in the checkpoint hour

Source Alignment

- `Advanced/lessons/lecture/Day6_Hour3_Advanced.md` → `## Learning objectives`

Why JSON Now, Not SQLite

Criterion	JSON today	SQLite next session
Human-readable	✓ easy to inspect	✗ binary format
Setup cost	✓ zero	requires schema DDL
Debug speed	✓ open in editor	requires tooling
Query power	limited	✓ SQL
Concurrent writes	limited	✓ supported

> "Good engineers start with the solution that best fits the milestone."

Architecture matters here:

- Strong service boundaries today make the SQLite swap **cheaper** tomorrow
- Stability beats novelty in a checkpoint week

Service-Side JSON Persistence

```python
from __future__ import annotations
import json
from dataclasses import asdict
from pathlib import Path


class StorageError(Exception):
    pass


class TrackerService:
    # ... existing CRUD methods ...

    def save_to_json(self, file_path: Path) -> None:
        data = [asdict(item) for item in self._items]
        try:
            file_path.parent.mkdir(parents=True, exist_ok=True)
            file_path.write_text(json.dumps(data, indent=2), encoding="utf-8")
        except OSError as error:
            raise StorageError(f"Unable to save: {error}") from error

    def load_from_json(self, file_path: Path) -> None:
        if not file_path.exists():
            self._items = []
            return
        try:
            raw = json.loads(file_path.read_text(encoding="utf-8"))
        except json.JSONDecodeError as error:
            raise StorageError(f"Invalid JSON: {error}") from error
        except OSError as error:
            raise StorageError(f"Cannot read file: {error}") from error
        self._items = [TrackerItem(**r) for r in raw]
```

Load on Startup

```python
from pathlib import Path
from tkinter import messagebox


class TrackerApp:
    def __init__(self, root, service):
        self.root = root
        self.service = service
        self.data_file = Path("data/tracker_items.json")
        self.selected_item_id = None
        self.status_text = tk.StringVar(value="Ready")
        self.name_var = tk.StringVar()
        self.category_var = tk.StringVar()
        self.status_var = tk.StringVar()

        self.build_ui()
        self.load_data_on_startup()   # load before first refresh
        self.refresh_items()

    def load_data_on_startup(self):
        try:
            self.service.load_from_json(self.data_file)
        except StorageError as error:
            messagebox.showerror("Load Error", str(error))
            self.set_status("Started with empty data — load error")
            return
        self.set_status(f"Loaded {len(self.service.list_items())} items")
```

persist_changes() Helper

Use a **single** helper so all three callbacks call the same save logic:

```
def persist_changes(self) -> bool:
    try:
        self.service.save_to_json(self.data_file)
    except StorageError as error:
        messagebox.showerror("Save Error", str(error))
        self.set_status("Save failed")
        return False
    self.set_status("Changes saved")
    return True
```

> ⚠️ Saving only after add, and forgetting update/delete,
> is the **most common persistence bug** at this stage.

The pattern for every mutation callback:

1. Do the domain action via service

CRUD Callbacks With Persistence Hooks

```
def on_add(self):
    try:
        item = self.service.add_item(**self.get_form_data())
    except ValidationError as error:
        messagebox.showerror("Validation Error", str(error))
        return
    self.persist_changes()
    self.refresh_items(selected_item_id=item.item_id)
    self.clear_form(keep_status=True)

def on_update(self):
    if self.selected_item_id is None:
        messagebox.showwarning("No Selection", "Select a record to update.")
        return
    try:
        item = self.service.update_item(self.selected_item_id,
                                        **self.get_form_data())
    except (ValidationError, NotFoundError) as error:
        messagebox.showerror("Update Error", str(error))
        return
    self.persist_changes()
    self.refresh_items(selected_item_id=item.item_id)

def on_delete(self):
    if self.selected_item_id is None:
        messagebox.showwarning("No Selection", "Select a record to delete.")
        return
    try:
        item = self.service.get_item(self.selected_item_id)
    except NotFoundError as error:
        messagebox.showerror("Delete Error", str(error))
        return
    if not messagebox.askyesno("Confirm Delete", f"Delete '{item.name}'?"):
        return
    self.service.delete_item(item.item_id)
    self.persist_changes()
    self.refresh_items()
    self.clear_form(keep_status=True)
    self.selected_item_id = None
```

Graceful JSON Error Handling

Three failure scenarios the app must survive:

Scenario	Expected behavior
File missing on first launch	Start with empty list; no crash
File corrupted or invalid JSON	Show `showerror` dialog; continue with empty list
Disk write failure on save	Show `showerror` dialog; data stays in memory

Never let a JSON error produce a raw traceback for the user.

```
except json.JSONDecodeError as error:
    raise StorageError(f"The data file is not valid JSON: {error}") from error
```

Recovery strategy for malformed file:

- Show error dialog

Optional: JsonTrackerStore for Stronger Separation

If the class is moving quickly, show a dedicated storage helper:

```
class JsonTrackerStore:
    def __init__(self, file_path: Path) -> None:
        self.file_path = file_path

    def save_items(self, items: list[TrackerItem]) -> None:
        payload = [asdict(item) for item in items]
        self.file_path.parent.mkdir(parents=True, exist_ok=True)
        self.file_path.write_text(json.dumps(payload, indent=2),
                                  encoding="utf-8")

    def load_items(self) -> list[TrackerItem]:
        if not self.file_path.exists():
            return []
        raw = json.loads(self.file_path.read_text(encoding="utf-8"))
        return [TrackerItem(**r) for r in raw]
```

Note — the checkpoint does not require this abstraction.

Usability Polish: Menu and Keyboard Shortcut

Polish is **not** decoration — it teaches users what the app is doing.

```
def build_menu(self) -> None:
    menubar = tk.Menu(self.root)

    file_menu = tk.Menu(menubar, tearoff=False)
    file_menu.add_command(label="Save",
                          command=self.persist_changes,
                          accelerator="Ctrl+S")
    file_menu.add_separator()
    file_menu.add_command(label="Exit", command=self.root.destroy)
    menubar.add_cascade(label="File", menu=file_menu)

    help_menu = tk.Menu(menubar, tearoff=False)
    help_menu.add_command(label="About", command=self.show_about)
    menubar.add_cascade(label="Help", menu=help_menu)

    self.root.config(menu=menubar)
    self.root.bind("<Control-s>",
```

Lab: Mini-Project Sprint

Time — 25–35 minutes

Required Tasks

1. Load records from JSON on app startup
2. Save after add, update, and delete
3. Handle a missing data file without crashing
4. Handle invalid JSON with a clear error message
5. Add **at least one** usability improvement

Choose at Least One Polish Feature

- File menu with Save and Exit
- Help/About dialog
- Status bar or confirmation

Common Pitfalls — Hour 23

⚠️ **Saving after add, but forgetting update/delete** — persistence is inconsistent
⚠️ **Crashing on malformed JSON** — always wrap `json.loads` in exception handling
⚠️ **Storing widget state instead of domain objects** — serialize `TrackerItem`, not UI details
⚠️ **Dropping stable IDs on save** — the JSON must preserve existing `item_id` values
⚠️ **Polishing visuals before save/load actually works** — a stylish app that loses data is a failed milestone

Quick Check

> Why must save happen after **update** and **delete**, not only after add?

Strong answer: "*The JSON file must reflect the current source of truth.*
Forgetting to save after update or delete creates a mismatch between what the user sees

Hour 24 — Checkpoint 3: GUI Milestone Demo

Checkpoint Overview

Expected outcome — a working GUI CRUD app wired to the service layer.

Grading focus (four pillars):

1. **UI works** — launch reliability and usable controls
2. **CRUD complete** — add, list, update, delete all through the interface
3. **Errors handled** — expected mistakes do not crash the app
4. **Code organization** — clean separation between UI and service responsibilities

Must-pass gates (any failure caps the score):

- Stable IDs for update and delete — including after restart and load
- Validation in service or core layer, not only in UI widget callbacks

Instructor Stance for This Hour

Use a **coaching evaluator** stance — rigorous on evidence, calm on process.

Opening language to read:

> "This checkpoint is a proof of working software and maintainable structure,
> not a perfection contest. I will score what you demonstrate.
> If something breaks, narrate your expected behavior and what you would inspect first.
> Good engineering communication still earns credit."

Score only **directly observed** behavior:

- "It worked earlier" — does not count
- "The code should do this" — does not count
- Explanation supports partial understanding credit but cannot replace execution

Canonical Demo Sequence

Prompt every learner with the same script:

> "Please run the canonical Hour 24 sequence now: **add** → **list** → **update** → **delete** → **restart** → **load**."

Then ask three standard questions:

1. "Show where your service layer performs validation."
2. "Show how selected rows map to stable IDs."
3. "Trigger one typical error path and show no-crash behavior."

Time discipline:

- Core flow target — **5 minutes**
- Q&A and architecture explanation — **2–3 minutes**
- Recovery overhead — **2 minutes maximum**

Must-Pass Gates

Gate 1 — Stable Identity for Update and Delete

Update and delete must target stable IDs, not row index, **including after restart and load**.

Fail indicators: wrong record changes after insert or sorting; behavior differs after reload.

Gate 2 — Service or Core Validation

Validation must exist in service or core layer. UI calls service and handles raised errors.

Fail indicators: service accepts invalid state when called directly; different UI paths bypass rules.

Gate 3 — Typical Error Path Resilience

Checkpoint Timing Crosswalk

Window	Required duration	Slot in Hour 24
Instructor talk points	10–20 min	Min 0–14: grading focus, rubric, must-pass gates
Live demo (instructor)	5–10 min	Min 14–21: fast checklist run of canonical flow
Hands-on lab	25–35 min	Min 21–52: supervised demo prep + rolling checks
Checkpoint 3 block	45–60 min	Entire hour — scoring and remediation routing

These are **not** four separate sessions — they are operational components of one

Checkpoint Rubric

Category	Points	Strong evidence
GUI launches and is operable	15	First launch succeeds; controls visible and labeled
CRUD through UI complete	30	Add, list, update, delete all work; refresh coherent
Persistence and restart load	20	Save, restart, reload confirm identical records
UI/core separation	20	Callbacks delegate to service; files explainable
Error handling and reliability	10	Typical error path shown without crash
Demo clarity and reflection	5	Design choices explained; next improvement

Scoring Decision Matrix

Status	Conditions	Next action
Pass	All deliverables observed; all three gates satisfied; no crash in typical use	Mark pass; give one stretch recommendation
Partial	Core works but one gate unresolved or one deliverable not fully demonstrated	Mark partial; assign targeted remediation + recheck deadline
Not yet	Multiple core failures; canonical flow incomplete; repeated crashes	Mark not yet; structured recovery plan + coaching block

Source Alignment

- `Advanced/lessons/lecture/Day6_Hour4_Advanced.md` → `### Decision matrix for final checkpoint status`

Triage Playbooks for Common Blockers

Blocker 1 — Wrong row updates after insert or reload

Questions: "What uniquely identifies a record in your model?" / "Where is that value stored in the UI list?"
Fix: persist IDs in JSON; bind selection to ID, not visual index; re-run canonical flow after restart.

Blocker 2 — Service bypasses validation

Questions: "Where is the authoritative validation function?" / "What error is raised for invalid data?"
Fix: move rule into service or model; UI handles raised error; keep lightweight UI pre-validation for UX only.

Blocker 3 — Crash on typical error path

Lab: Demo Prep Sprint

Time — 25–35 minutes (rolling checkpoint checks during this window)

Learner Tasks

1. Run the full canonical sequence: add → list → update → delete → restart → load
2. Fix any blockers from instructor circulation prompts
3. Prepare a one-sentence explanation of where the service layer lives
4. Prepare a one-sentence answer for: "How does update find the correct record?"

Instructor Circulation Prompts

- "Show me where the selected record ID is captured."
- "What layer enforces non-empty name validation?"
- "If no row is selected and the user clicks Update, what happens?"
- "If the JSON file

Closing Quick Check

Ask every learner before the session ends:

> **"If you had one more hour, what would you improve first in your GUI?"**

Collect answers verbally or in chat.

Prioritize responses that mention:

- Reliability and error handling
- Architecture boundaries
- User trust (persistence, confirmations, feedback messages)

Over responses that mention only visual or cosmetic changes.

Checkpoint 3 is complete when every learner has a score decision and a concrete next action.

Session 6 Summary — What We Built

Four Hours, One Cohesive Story

Hour	Theme	Key Deliverable
21	Update and delete workflows	Stable-ID CRUD round trip in Tkinter
22	Controller separation	`TrackerApp` class; thin callbacks; clean module graph
23	Persistence and polish	JSON load/save; graceful error handling; one polish feature
24	Checkpoint 3 demo	End-to-end evidence: add → list → update → delete → restart → load

What's Next — Session 7 (Hours 25–28)

From Objects to Tables

- **SQLite CRUD** with the `sqlite3` module
- Writing safe parameterized queries (no string concatenation)
- Row mapping: converting database rows back to `TrackerItem` objects
- Database hardening: schema migration and integrity checks
- Swapping JSON storage for SQLite in the same `TrackerApp` architecture

> Because the service layer is already the gateway for all operations,
> the storage swap will touch the service and storage modules —
> the UI layer stays largely stable.

Source Alignment